05 · Reconciliation

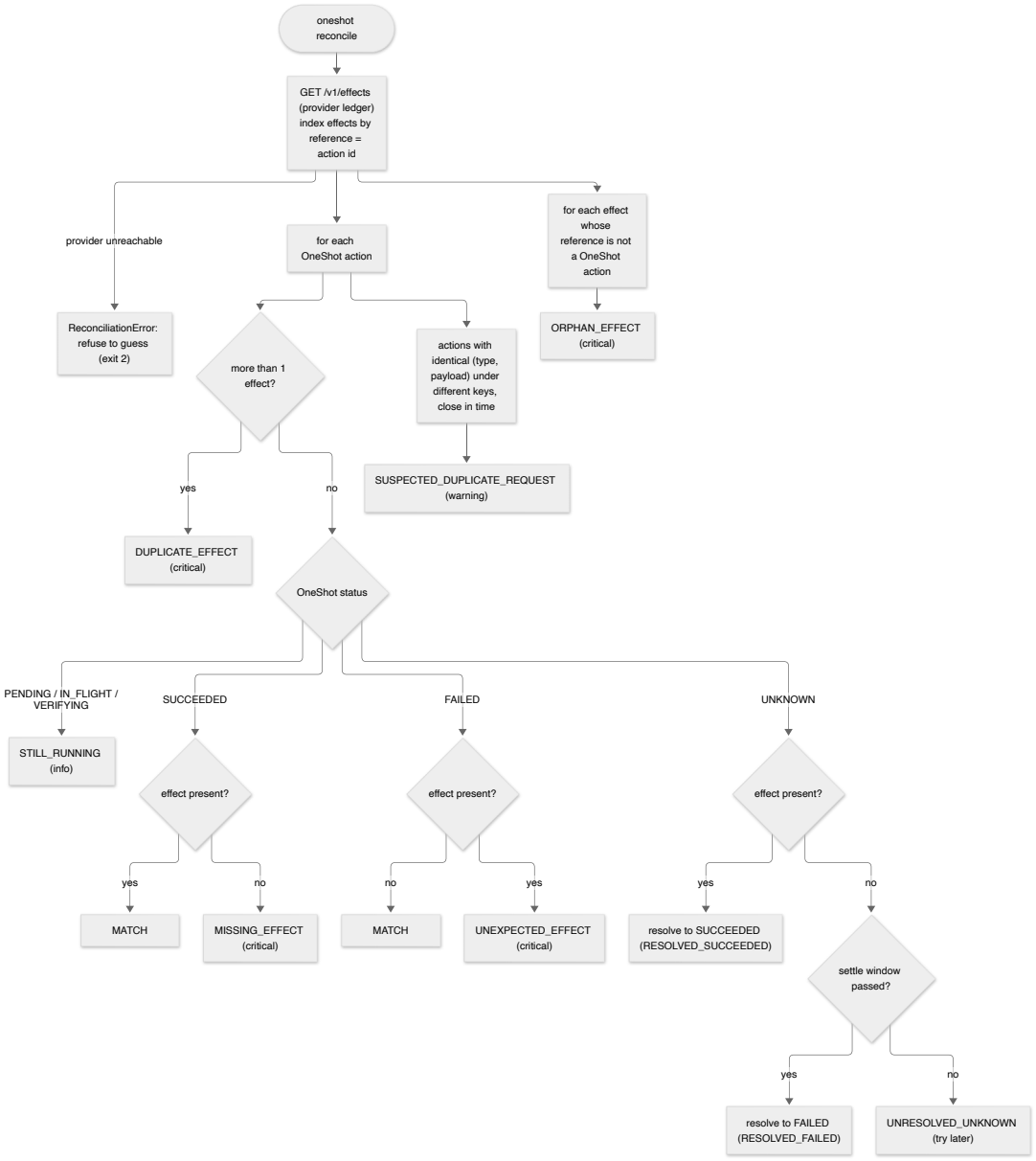
Compare what OneShot believes with what the provider actually did, resolve what can be resolved, and flag the rest.

The problem

Every mechanism so far runs *inside* OneShot and relies on OneShot's own records. Records drift from reality: a database restored from a backup, a bug, a duplicate that slipped through, an action left UNKNOWN while the provider was down. Banks and payment companies close this gap the same way: they **regularly compare their ledger with the counterparty's ledger, line by line**. That process is called reconciliation.

The essential property is that the comparison uses an **independent record**. OneShot's reconciler fetches the provider's ledger over HTTP from the provider process (`ProviderClient.ledger()` → `GET /v1/effects`). That ledger is a separate SQLite file that only the provider writes. OneShot reading its own notes would prove nothing.

What is compared



Finding	Severity	Automatic action	Typical cause
`MATCH`	ok	none	everything worked
`STILL_RUNNING`	info	none	worker has not finished
`RESOLVED_SUCCEEDED`	resolved	UNKNOWN → SUCCEEDED	provider was down during verification
`RESOLVED_FAILED`	resolved	UNKNOWN → FAILED	provider crashed before applying
`UNRESOLVED_UNKNOWN`	warning	none (run again later)	still inside the settle window
`SUSPECTED_DUPLICATE_REQUEST`	warning	none	agent retried with a new key, *or* a genuine repeat
`DUPLICATE_EFFECT`	critical	none	slow non-idempotent provider, misconfigured retention

Finding	Severity	Automatic action	Typical cause
`MISSING_EFFECT`	critical	none	bug, provider data loss, contract violation
`UNEXPECTED_EFFECT`	critical	none	provider applied after a "permanent" rejection
`ORPHAN_EFFECT`	critical	none	OneShot DB lost or restored from backup; a client bypassed OneShot

`oneshot reconcile` exits with code 1 when any critical finding exists, so it can run from cron or CI and page someone.

Why only UNKNOWN is changed automatically

UNKNOWN exists precisely to be resolved by better information, and the provider's ledger *is* that information. Terminal states are different: SUCCEEDED or FAILED has already been reported to the agent, which may have acted on it. Silently flipping a FAILED to SUCCEEDED would hide an incident: the agent may have retried with a new key and charged twice. So mismatches on terminal states are **flagged for a human, never rewritten**. Fixing money (refunds, compensating actions) is a business decision.

`RESOLVED_FAILED` is only issued after the settle window has passed since the last send. This is the same assumption verification uses (doc 02): before then, "absent" could just mean "not yet". Every resolution goes through `ActionStore.resolve()`, so it is audited with actor `reconciler` and the evidence (effect ids).

How the code does it

What	Where
Fetch the independent ledger	`oneshot/provider_client.py`: `ProviderClient.ledger`
Classify per action	`oneshot/reconciler.py`: `Reconciler._check_action`
Resolve UNKNOWN (with settle rule)	`Reconciler._resolve_unknown` → `ActionStore.resolve`
Orphans	`Reconciler.run` (effects whose reference is unknown)
Content-duplicate heuristic	`Reconciler._suspected_duplicate_requests` (same fingerprint within 300 s)
Dry run / JSON / exit code	`oneshot/cli.py`: `cmd_reconcile` (`--dry-run`, `--json`, `--all`)
API	`POST /reconcile?apply=true`

Observed: reconciler and API tests

```
$ python -m pytest -v tests/test_reconciler.py tests/test_api.py
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

==================================== test session starts =====================================
collecting ... collected 10 items

tests/test_reconciler.py::test_each_mismatch_is_classified PASSED                          [ 10%]
tests/test_reconciler.py::test_unknown_is_only_resolved_failed_after_the_settle_window PASSED [ 20%]
tests/test_reconciler.py::test_identical_requests_under_different_keys_are_only_a_warning PASSED [ 30%]
tests/test_api.py::test_submit_then_replay_same_key PASSED                                 [ 40%]
```

```
tests/test_api.py::test_key_reuse_with_different_payload_is_422 PASSED                    [ 50%]
tests/test_api.py::test_idempotency_key_is_required PASSED                                [ 60%]
tests/test_api.py::test_invalid_payload_and_unknown_type_are_400 PASSED                   [ 70%]
tests/test_api.py::test_payload_is_normalised_before_fingerprinting PASSED                [ 80%]
tests/test_api.py::test_get_action_has_timeline PASSED                                    [ 90%]
tests/test_api.py::test_resolve_only_unknown PASSED                                       [100%]

======================================== 10 passed in 0.20s ========================================
```

Observed: provider outage → UNKNOWN → resolved (demo)

```
$ python -m oneshot demo provider-crash-unknown
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────────── OneShot demo ────────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-fj82sh8h

1/1  provider-crash-unknown: Provider crashes right after applying, and stays down ──────────────────
  > fault armed at the provider: crash_after_apply on /v1/messages
  > agent POST /actions Idempotency-Key=agent-notify-2003 -> HTTP 201 act_a5e50870ea40: message to
+15550100: 'Your order has shipped'
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  Provider's side of the story (its log):
    POST /v1/messages ref=act_a5e50870ea40 key=- mode=crash_after_apply -> APPLIED eff_00001, then
CRASHING (exit 3)
  [PASS] action ended UNKNOWN while the provider was down
  > provider restarted (its ledger file survived the crash)
  > run reconciliation against the provider's ledger
Reconciliation (applied): 1 actions vs 1 provider effects
┌──────────────────────┬──────────────────┬──────────────────┬──────────────────────────────────────┐
│ finding              │ action           │ provider effects │ details                              │
├──────────────────────┼──────────────────┼──────────────────┼──────────────────────────────────────┤
│ RESOLVED_SUCCEEDED   │ act_a5e50870ea40 │ eff_00001        │ UNKNOWN -> SUCCEEDED: provider has   │
│                      │                  │                  │ eff_00001                            │
└──────────────────────┴──────────────────┴──────────────────┴──────────────────────────────────────┘
                    0 action(s) matched the provider exactly (hidden; use --all)
  OneShot's record (event timeline):
  act_a5e50870ea40  message to +15550100: 'Your order has shipped'  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87200, intent recorded before
sending
    +  0.07s IN_FLIGHT -> VERIFYING  connection dropped before a response arrived; outcome unknown
and provider does not deduplicate -> check provider's record at settle time (+1.50s) before deciding
    +  1.60s VERIFYING -> VERIFYING  could not check provider (lookup failed: ConnectError); will
try again in 0.30s (check 1/3)
    +  1.90s VERIFYING -> VERIFYING  could not check provider (lookup failed: ConnectError); will
try again in 0.60s (check 2/3)
    +  2.51s VERIFYING -> UNKNOWN  could not check provider after 3 tries (lookup failed:
ConnectError); outcome cannot be determined -> needs investigation / reconciliation
    +  2.62s UNKNOWN -> SUCCEEDED  resolved by reconciler: provider ledger shows effect eff_00001
    resolution: reconciler: provider ledger shows effect eff_00001
  Provider's independent ledger:
    act_a5e50870ea40: 1 effect(s) ['eff_00001']
  [PASS] reconciliation resolved it
  [PASS] act_a5e50870ea40 is SUCCEEDED and provider ledger has 1 effect(s) (observed: SUCCEEDED, 1)
  Lesson: Ambiguous outcome + no way to check = UNKNOWN, an honest 'we do not know'. When the
provider is back, reconciliation reads its ledger and resolves UNKNOWN to SUCCEEDED.

Scorecard (every expectation checked against the
provider's real ledger)
┌──────────────────────┬────────┬──────┬────────┐
│ scenario             │ checks │ time │ result │
├──────────────────────┼────────┼──────┼────────┤
```

```
| provider-crash-unknown |    3/3 | 2.6s | PASS   |
```

Observed: OneShot's database wiped, provider evidence survives (demo)

```
$ python -m oneshot demo database-wiped
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────────── OneShot demo ────────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-6xsuqz1i

1/1  database-wiped: OneShot's database is lost ──────────────────────────────────────────────────────
  > agent POST /actions Idempotency-Key=agent-order-1007 -> HTTP 201 act_fb98e0261b64: charge
cus_ada 42.00 USD
  > agent POST /actions Idempotency-Key=agent-notify-2005 -> HTTP 201 act_423ff5e62efb: message to
+15550100: 'Your order has shipped'
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  OneShot's record (event timeline):
  act_fb98e0261b64  charge cus_ada 42.00 USD  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87211, intent recorded before
sending
    +  0.07s IN_FLIGHT -> SUCCEEDED  provider confirmed: applied as eff_00001
  act_423ff5e62efb  message to +15550100: 'Your order has shipped'  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.07s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87211, intent recorded before
sending
    +  0.07s IN_FLIGHT -> SUCCEEDED  provider confirmed: applied as eff_00002
  Provider's side of the story (its log):
    POST /v1/charges ref=act_fb98e0261b64 key=act_fb98e0261b64 mode=success -> APPLIED eff_00001,
responding 201
    POST /v1/messages ref=act_423ff5e62efb key=- mode=success -> APPLIED eff_00002, responding 201
  Provider's independent ledger:
    act_fb98e0261b64: 1 effect(s) ['eff_00001']
    act_423ff5e62efb: 1 effect(s) ['eff_00002']
  OneShot's database files deleted. Creating an empty one.
  > run reconciliation against the provider's ledger
Reconciliation (applied): 0 actions vs 2 provider effects
┌─────────────────┬──────────────────┬─────────────────┬──────────────────────────────────────────┐
│ finding         │ action           │ provider effects │ details                                  │
├─────────────────┼──────────────────┼─────────────────┼──────────────────────────────────────────┤
│ ORPHAN_EFFECT   │ act_fb98e0261b64 │ eff_00001       │ provider applied charge eff_00001        │
│                 │                  │                 │ (reference=act_fb98e0261b64) but OneShot │
│                 │                  │                 │ has no such action (database lost/restored │
│                 │                  │                 │ from backup, or a client bypassed OneShot) │
│ ORPHAN_EFFECT   │ act_423ff5e62efb │ eff_00002       │ provider applied message eff_00002       │
│                 │                  │                 │ (reference=act_423ff5e62efb) but OneShot │
│                 │                  │                 │ has no such action (database lost/restored │
│                 │                  │                 │ from backup, or a client bypassed OneShot) │
└─────────────────┴──────────────────┴─────────────────┴──────────────────────────────────────────┘
                    0 action(s) matched the provider exactly (hidden; use --all)
  [PASS] both real effects are reported as orphans
  Lesson: The provider's ledger is independent evidence. Even with OneShot's records gone,
reconciliation shows every effect that really happened as an ORPHAN that needs attention.

Scorecard (every expectation checked
against the provider's real ledger)
┌─────────────────┬────────┬──────┬────────┐
│ scenario        │ checks │ time │ result │
├─────────────────┼────────┼──────┼────────┤
│ database-wiped  │    1/1 │ 0.2s │ PASS   │
└─────────────────┴────────┴──────┴────────┘
```

Protects against: UNKNOWN actions lingering forever; duplicates going unnoticed; OneShot and the provider silently disagreeing; losing track of effects after OneShot data loss; agents told the wrong outcome without anyone noticing.

Cannot protect against: a wrong provider ledger (it is trusted as the source of truth); effects that do not carry OneShot's reference (they appear as orphans with no link back); undoing anything. Detection happens after the fact, so the gap between an incident and its detection is the reconciliation interval. It also does not scale as written: it reads the *whole* ledger every run (fine for a demo; real systems page through a cursor such as `created_after` and reconcile in windows, see ROADMAP).

Design decisions and alternatives

- **Pull the provider's ledger over the public API**, as you would with a real provider's "list charges" endpoint or a daily settlement file. The rejected alternative, reading the provider's database file directly, is impossible with a real third party and would blur the "independent evidence" boundary this design depends on.
- **Severity levels instead of a pass/fail.** Operators need to know what to look at first; `STILL_RUNNING` is not an incident, `DUPLICATE_EFFECT` is.
- **Heuristic duplicate detection is a warning, never a block**, because identical requests can be legitimate (doc 03).

Self-check questions

1. Why must the reconciler read the provider's ledger through the provider's API rather than OneShot's database?
2. An action is FAILED in OneShot but the provider has an effect. Why doesn't the reconciler just correct it to SUCCEEDED?
3. An UNKNOWN action has no provider effect, and the last send was 1 s ago (settle window 5 s). What does reconciliation report, and why?
4. What does an `ORPHAN_EFFECT` tell you, and what is the first thing to check?
5. Why does `oneshot reconcile` return a non-zero exit code on critical findings?

Answers. (1) Reconciliation is only meaningful against an independent record; OneShot's own data is exactly what might be wrong. (2) The agent was already told FAILED and may have acted on it (for example by resubmitting); flipping it would hide a real incident that needs a human. (3) `UNRESOLVED_UNKNOWN` : the original request might still land, so "absent" proves nothing yet. (4) The provider applied something OneShot has no record of: check for data loss or restore in OneShot, or another client using OneShot's reference. (5) So that scheduled runs (cron/CI) can alert automatically.